File Carving

Scenario Overview

The security team detected an unusual file leaving the network and could not immediately determine whether it contained sensitive information. The file appeared visually as a solid green image, raising suspicion that additional data might be embedded within it. Your task is to analyze the file, identify its true format, determine whether it contains hidden content, and recover any concealed information or flags.

Tasks

1. **Identify the file format**
    - Download the suspicious file labeled green_file.
    - Use the file command to determine the actual file type behind the green appearance.
2. **Analyze the file for embedded content**
    - Run binwalk on the file to detect any additional files or binary objects packaged within it.
    - Count how many extractable components are present in the binary blob.
3. **Extract hidden files**
    - Review the binwalk manual if needed to understand extraction options.
    - Use binwalk --extract or --dd to carve out embedded files from the binary.
    - Inspect the extracted directory for non-image or unusual files.
4. **Recover the hidden flag**
    - Extract the archive or container file (e.g., using tar xvf).
    - Navigate through the extracted structure to locate any text files or flag directories.
    - Read the contents of the flag file to recover the hidden flag.

1. This file initially looks like something green, what's the file format of this green file?

Download the file `green_file` and navigate to your downloads folder `~/Downloads` followed by command `file green_file` to verify the type of file.

Answer: PNG

```
┌──(kali㉿kali)-[~/Downloads]
└─$ file green_file
green_file: PNG image data, 63 x 36, 8-bit/color RGBA, non-interlaced

┌──(kali㉿kali)-[~/Downloads]
└─$
```

2. How many files can be extracted from the binary blob?

Run `binwalk green_file` to view any other files that are packaged into the file. We now see the number of files that can be extracted.

Answer: 6

```
┌──(kali㉿kali)-[~/Downloads]
└─$ binwalk green_file

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             PNG image, 63 x 36, 8-bit/color RGBA, non-interlaced
3243          0xCAB           gzip compressed data, from Unix, last modified: 2017-02-14
 05:32:27
3605          0xE15           PNG image, 24 x 24, 8-bit/color RGBA, non-interlaced
3818          0xEEA           PNG image, 24 x 24, 8-bit/color RGBA, non-interlaced
4040          0xFC8           PNG image, 24 x 24, 8-bit/color RGBA, non-interlaced
4264          0x10A8          PNG image, 24 x 24, 8-bit/color RGBA, non-interlaced

┌──(kali㉿kali)-[~/Downloads]
└─$
```

3. What is the hidden flag in the file?

Run command `man binwalk` to become familiar with the tool. The command displays the manual pages for the binwalk tool. It shows what binwalk is, what it does, and all available command-line options. Type `q` to quit.

```
Session  Actions  Edit  View  Help
BINWALK(1)                    User Commands                    BINWALK(1)

NAME
       binwalk - tool for searching binary images for embedded files and executable
       code

SYNOPSIS
       binwalk [OPTIONS] [FILE1] [FILE2] [FILE3] ...

DESCRIPTION
       Binwalk v2.3.2+dcb1403 Craig Heffner, ReFirmLabs https://github.com/ReFirm-
       Labs/binwalk

   Signature Scan Options:
       -B, --signature
              Scan target file(s) for common file signatures

       -R, --raw=<str>
              Scan target file(s) for the specified sequence of bytes

       -A, --opcodes
              Scan target file(s) for common executable opcode signatures

       -m, --magic=<file>
              Specify a custom magic file to use

       -b, --dumb
              Disable smart signature keywords

       -I, --invalid
              Show results marked as invalid

       -x, --exclude=<str>
              Exclude results that match <str>

       -y, --include=<str>
              Only show results that match <str>

   Extraction Options:
       -e, --extract
              Automatically extract known file types

       -D, --dd=<type[:ext[:cmd]]>
              Extract <type> signatures (regular expression), give the files an exten-
              sion of <ext>, and execute <cmd>
 Manual page binwalk(1) line 1 (press h for help or q to quit)
```

Runn the extraction command `binwalk --extract --dd “png:png” green_file`

```
aa-load
aa-remove-unknown
aa-status
aa-teardown
ab
abort
ABORT
abs
accept
accept4
access
access.conf
accessdb
acct
acl
acorn
acos
acosf
acosh
acoshf
acoshl
acosl
activate-global-python-argcomplete
acyclic
addgnupghome
addgroup
add_key
┌──(kali㉿kali)-[~/Downloads]
└─$ man binwalk

┌──(kali㉿kali)-[~/Downloads]
└─$ binwalk --extract --dd "png:png" green_file

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             PNG image, 63 x 36, 8-bit/color RGBA, non-interlaced
3243          0xCAB           gzip compressed data, from Unix, last modified: 2017-0
 05:32:27
3605          0xE15           PNG image, 24 x 24, 8-bit/color RGBA, non-interlaced
3818          0xEEA           PNG image, 24 x 24, 8-bit/color RGBA, non-interlaced
4040          0xFC8           PNG image, 24 x 24, 8-bit/color RGBA, non-interlaced
4264          0x10A8          PNG image, 24 x 24, 8-bit/color RGBA, non-interlaced


┌──(kali㉿kali)-[~/Downloads]
└─$
```

Run `ls` to view the files and you'll see `green_file.extracted` Here we see the file CAB a non-image file.

```
┌──(kali㉿kali)-[~/Downloads]
└─$ _green_file.extracted

┌──(kali㉿kali)-[~/Downloads/_green_file.extracted]
└─$ cab
```

Run the command `tar xvf` CAB to view the archives.

```
┌──(kali㉿kali)-[~/Downloads/_green_file.extracted]
└─$ tar xvf CAB
flags/
flags/flags.txt

┌──(kali㉿kali)-[~/Downloads/_green_file.extracted]
└─$
```

To view the contents to search for the fag I ran `cat flags/flags.txt`

Answer: SKY-RWCI-4291